

CONTACT

+82-10-2627-1356

dmsgp2627@naver.com

github.com/eeeunhey

EDUCATION

고려사이버대학교

2026.03 – (편입재학)

AI 데이터과학부

성남 폴리텍 하이테크 과정

2024.01 – 2024.12

Java (Spring/Boot/MyBatis/JPA)

Python (LLM / Object Detection)

KDT 국비 과정 수료

2023.03 – 2023.09

C/Java/Python 기초, 실습

한국방송통신대학교 졸업

2016.02 – 2021.02

식품영양학과

KEY SKILLS

Java | Python | SQL | Spring | Spring Boot | MyBatis | JPA | React | Vite | TailwindCSS | LLM | Object Detection | Git | Docker | .NET C# WinForms

CERTIFICATIONS

컴퓨터활용능력 1급, 2급

ITQ A등급 (한글, 엑셀, 파워포인트)

GTQ 1급, 2급

자동차운전면허 1종 보통

문제에서 배움을 찾고 성장하는 김은혜 입니다

JUNIOR DEVELOPER (WEB · BACKEND · AI)

Portfolio

PROFESSIONAL PROFILE

저는 새로운 시도와 도전을 통해 문제를 발견하고 해결하며 성장해왔습니다.
작은 단위로 나누어 해결하고 개선을 반복한 경험은 저를 더 나은 방향으로 이끌었고,
앞으로도 이런 태도를 바탕으로 계속 발전해 나가겠습니다.

EXPERIENCE

솔루션팀 · 머신 비전 파트

2024.02 – 2024.11

- 비전 셋팅 및 카메라 파라미터 조정 지원
- C# WinForms UI 일부 수정 및 테스트
- 테스트 결과 정리 및 개선 사항 보고

쿠팡 물류 · 전산 / 관리

2019.03 – 2021.11

- 단기 인력 배치와 스케줄 조율
- 출고 전산 흐름 점검 및 마감 관리
- 마감 시점의 소통·조율 업무

PROJECTS

Velin — AI Interview Prep

2025.08–2025.09 (개인)

ALBAIT — 콘솔 구인·구직 실습

2025.04–2025.05 (개인)

Interview Simulator

2025.09–2025.11 (팀)

Ascend — 이력서 코치 웹

2025.10–2025.12 (개인)

ResumeBuilder — 이력서 작성 지원

2025.10–2025.11 (개인)

BalanceEat — AI 식단 관리

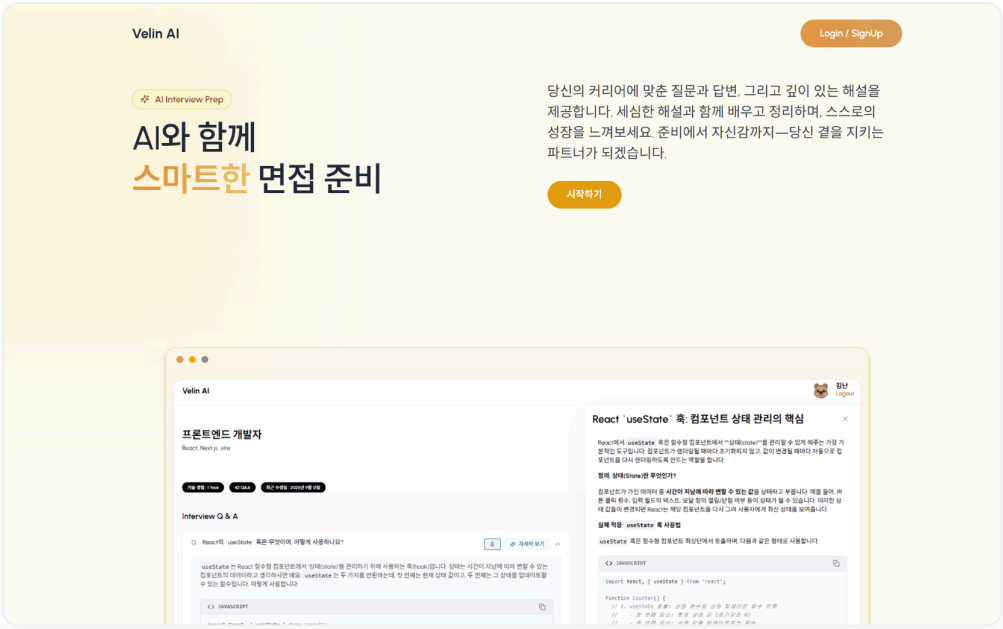
2025.10–2025.11 (팀)

Daily Emotion Log — 감정 기록

2025.12 (팀)

Velin — AI Interview Prep

2025.08–2025.09 (개인) · Vite·React·Tailwind / Node·Express / Recharts / TanStack Query / FFmpeg(WASM)



개요

면접 질문·근거·체크리스트 자동 생성과 멀티모달(영상/텍스트) 분석 기반 세션·질문 단위 리포트를 제공합니다.

개발 기간/인원

2025.08–2025.09 (6주) · 개인

기획 의도

- 이론 학습만으로는 협업·운영 관점에서 부족함을 느꼈습니다
- “함께 일하고 싶은 동료라면 어떤 질문과 근거가 필요할까?”라는 문제의식에서 출발했습니다
- 질문·참고 자료·체크리스트를 한눈에 제공하는 실무형 도구를 기획했습니다

핵심기능

- LLM 기반 질문/질문에 대한 근거 이론 / 코드 예시 제공
- IT 직군별 실무 질문 생성 후 제공
- 직군별 생성 질문 후 질문 추가 생성

기술스택

React · Vite · Tailwind · Node·Express · MongoDB

성과·배운 점

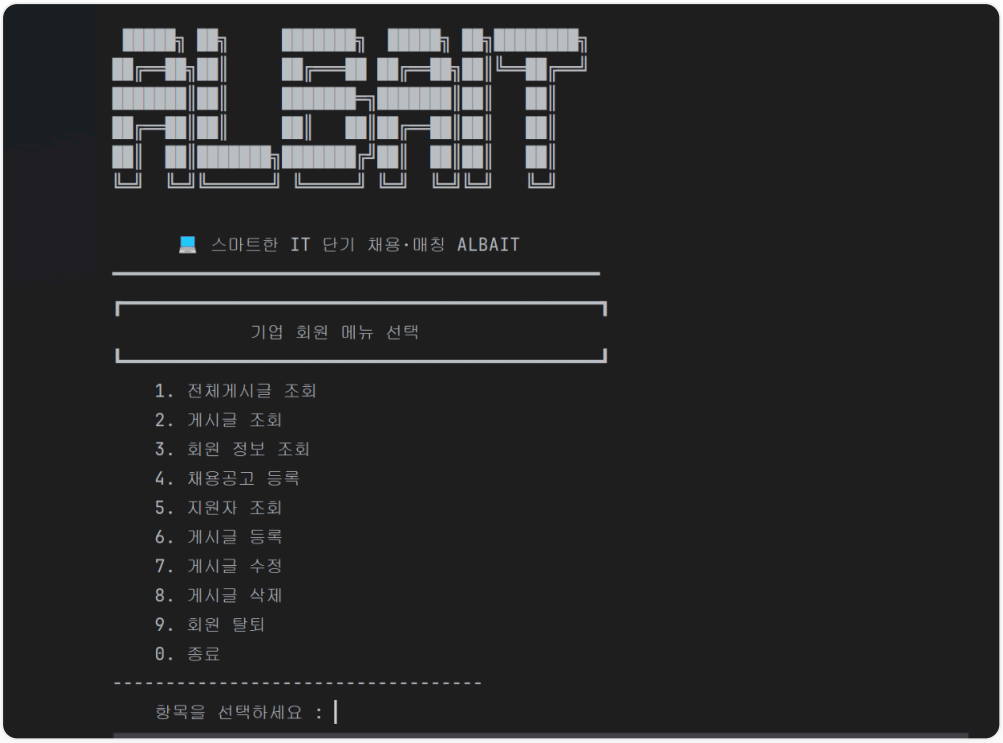
- ✓ **프롬프트 템플릿 설계** 과정을 통해 AI 출력 제어·일관성 유지의 중요성을 배움
- ✓ **React–Express–MongoDB 연동**으로 프론트와 백엔드, DB 연결 구조를 이해
- ✓ **JWT 인증과 파일 업로드**를 적용하며 보안과 데이터 흐름 관리 경험 습득

참고 링크

https://github.com/eeeunhey/AI_InterviewPrep

ALBAIT — 콘솔 구인·구직 사이트 구현

2025.04–2025.05 (개인) · Java · Oracle · JDBC · DAO/VO



개요

개인/기업 회원관리, 공고/지원/상태변경, 관리자 통계를 제공하는 콘솔 기반 프로젝트.

개발 기간/인원

2025.04–2025.05 (개인)

기획 의도

- 단순 CRUD 예제로는 실제 구인·구직 서비스 구조를 충분히 이해하기 어렵다고 느꼈습니다
- 회원·공고·지원 흐름을 직접 설계하여 실제 구직 과정을 콘솔 환경에서 구현했습니다
- DB 설계, DAO/VO, 트랜잭션 처리 등 실무 아키텍처 패턴을 학습하기 위해 프로젝트 진행 하였습니다

핵심기능

- 개인/기업 회원, 공고 등록/지원, 지원 상태 변경
- 관리자 통계(사용자/지원 현황) 조회

기술스택

Java · Oracle · JDBC · DAO/VO

성과·배운 점

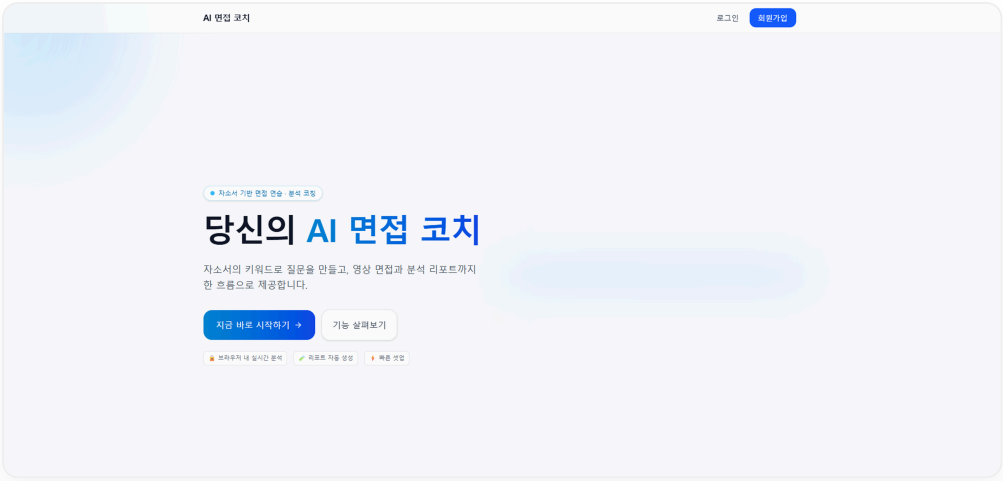
- ✓ **인덱스와 실행계획 분석**을 통해 SQL 성능 최적화 원리 학습
- ✓ **DAO/Service 분리**로 계층형 아키텍처 설계의 중요성 이해
- ✓ **예외 처리와 트랜잭션 관리**를 통해 안정적인 데이터 흐름 관리 방법을 습득
- ✓ **회원–공고–지원 흐름**을 구현하며 실제 구직 서비스 구조를 콘솔 환경에서 모사하는 경험
- ✓ **데이터베이스 모델링과 DAO/VO 패턴**을 통해 실무형 서비스 설계 감각을 익힘

참고 링크

https://github.com/eeeunhey/AlbaitProject

Interview Simulator — AI 모의 면접

2025.09–2025.11 (팀) · React · TypeScript · Recharts · WebRTC · Node.js · Express · MongoDB



개요

AI 면접 질문 생성부터 STT 기반 답변 분석, 감정·집중도 시각화, 세션 리포트까지 제공하는 모의 면접 웹 서비스입니다.

개발 기간/인원

2025.09–2025.11 (3개월) · 팀 프로젝트 / 담당: Frontend

역할

- 질문 생성 UI 설계 및 컨트롤 플로우 구현
- Recharts 기반 감정/집중도 시각화 컴포넌트 제작
- WebRTC 화상 면접 지원 연동
- 세션 리포트 페이지 구현

핵심기능

- AI 기반 면접 질문 자동 생성
- STT(Speech-to-Text) 실시간 답변 인식
- 감정/집중도 분석 시각화 (Recharts)
- WebRTC 기반 화상 인터뷿° 지원
- 백그라운드 업로드 및 세션 리포트

기술스택

React · TypeScript · Node.js · MongoDB · WebRTC · Recharts

성과·배운 점

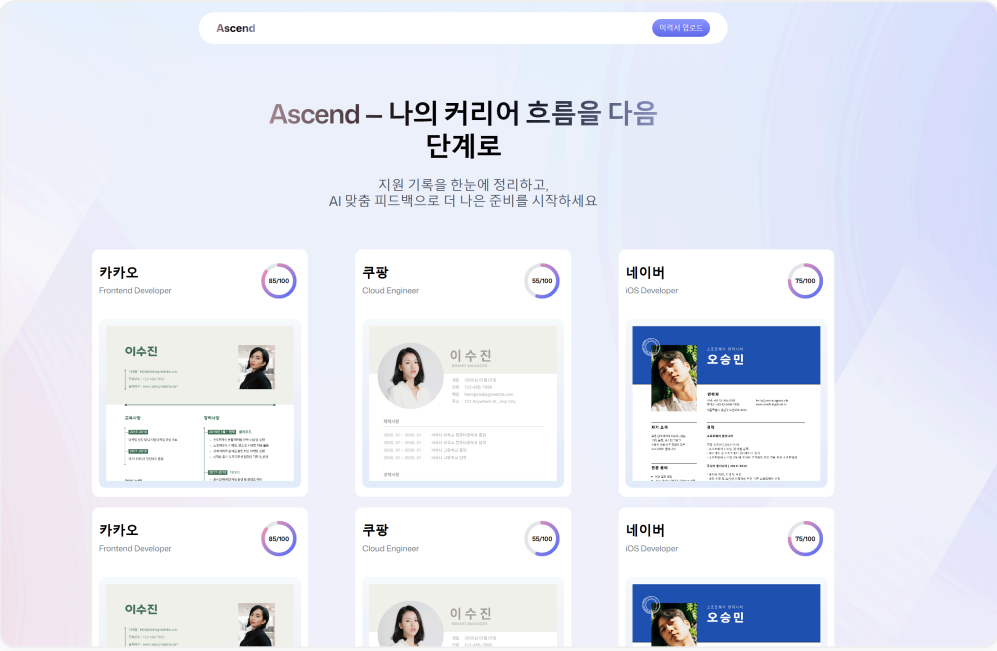
- ✓ **WebRTC** 화상 스트림 연동으로 P2P 통신 구조 이해
- ✓ **Recharts** 로 실시간 데이터 시각화 컴포넌트 설계 경험
- ✓ 팀 협업에서 정해진 인터페이스 기반 고첨합 역량 향상

참고 링크

https://github.com/eeeunhey/MockInterview

Ascend — AI 이력서 코치 웹

2025.10–2025.12 (개인) · TypeScript · React · Vite · Tailwind CSS · Zustand · React Query · Puter.js



개요

이력서 PDF를 업로드하면 AI가 섹션별 점수·피드백·키워드 개선안을 제시하는 이력서 코칭 웹 서비스입니다.

개발 기간/인원

2025.10–2025.12 (3개월) · 개인

핵심기능

- 이력서 PDF 업로드 및 텍스트 추출 (Puter.js)
- AI 기반 항목별 점수 및 피드백
- ATS 키워드 맵 시각화
- 섹션별 문구 개선 제안
- Zustand 전역 상태 관리
- React Query 비동기 처리

기술스택

React · TypeScript · Vite · Tailwind · Zustand · React Query

성과·배운 점

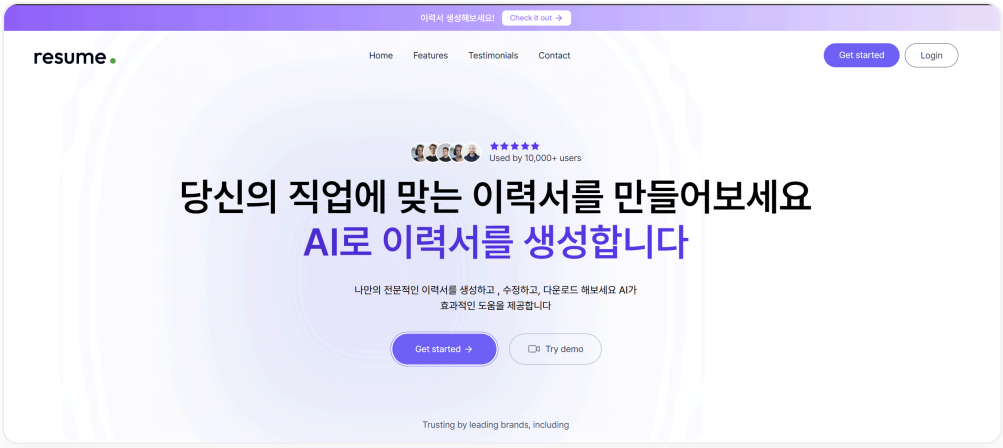
- ✓ **Zustand** 전역 스토어로 상태 일관성 확보 및 불필요한 리렌더 최소화 경험
- ✓ **React Query**로 비동기 데이터 쪠싱과 로딩/에러 상태 관리 학습
- ✓ PDF 텍스트 추출과 AI 연동 플로우 설계 경험

참고 링크

https://github.com/eeeunhey/Ascend

ResumeBuilder — 이력서 작성 지원 웹

2025.10–2025.11 (개인) · TypeScript · React · Vite · State Management



개요

구직자가 항목별 가이드에 따라 이력서를 작성하고, 실시간으로 미리보기 가능한 이력서 빌더 웹 앱입니다.

개발 기간/인원

2025.10–2025.11 (2개월) · 개인

핵심기능

- 섹션별(경력/학력/스킬/프로젝트) 구조화된 입력 폼
- 실시간 이력서 미리보기 패널
- 상태 관리 기반 입력 데이터 유지
- UX 중심의 입력 가이드 메시지

기술스택

React · TypeScript · Vite

성과·배운 점

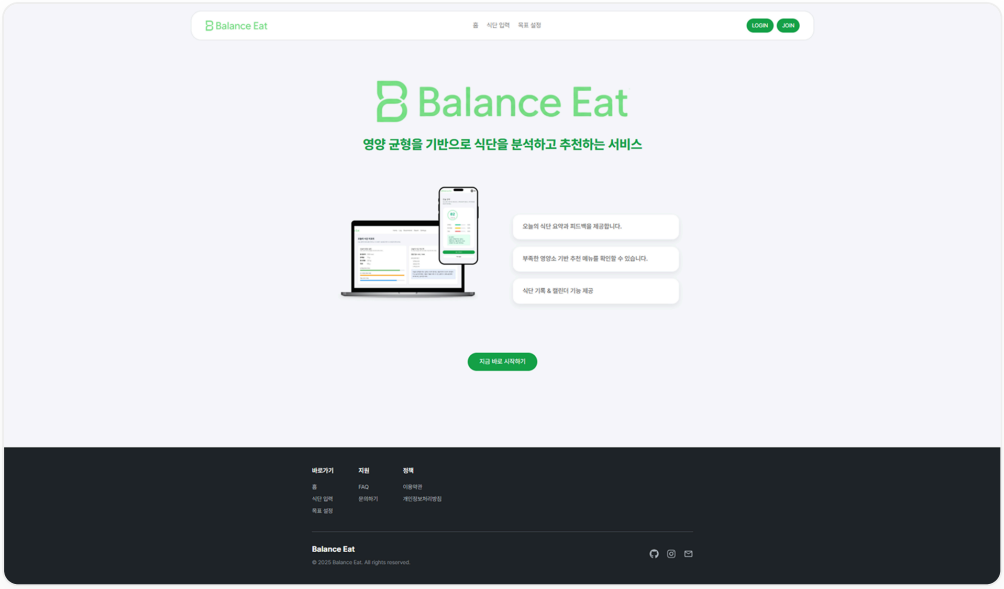
- ✓ React 상태와 폼 핸들링을 같이 설계하는 구조 학습
- ✓ 데이터 입력 흐름과 UI 갱신 효율 최적화 경험
- ✓ 사용자 열람을 고려한 UX 응용 설계 연습

참고 링크

https://github.com/eeeunhey/ResumeBuilder

BalanceEat — AI 식단 관리

2025.10–2025.11 (팀) · React · TypeScript · Gemini AI API · Nutrition API · Local Storage · Figma



개요

음식 섭취 기록과 AI 식단 분석을 통해 균형잡힌 식단 관리를 도와주는 웹 서비스입니다.

개발 기간/인원

2025.10–2025.11 (2개월) · 4인 팀 / 담당: PO / Frontend

핵심기능

- Nutrition API 기반 음식 영양 정보 검색
- Gemini AI를 활용한 맞춤 식단 추천
- 섭취량 시각화 (칼로리/영양소 차트)
- 식단 기록 & 날짜별 히스토리
- Local Storage 기반 데이터 영속화

기술스택

React · TypeScript · Gemini AI · Nutrition API · Figma

성과·배운 점

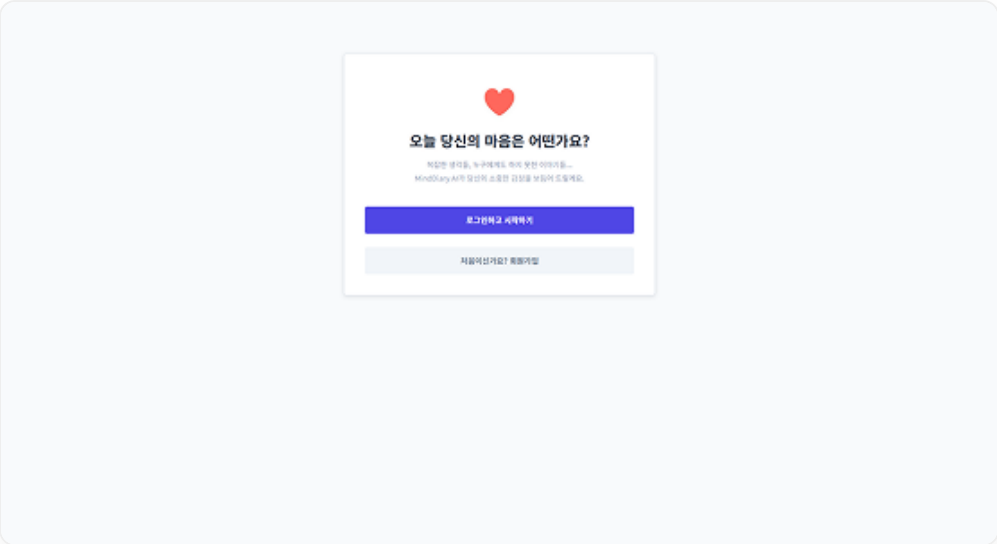
- ✓ **PO 역할**에서 요구사항 정리뿐 아니라 전체 진행 흐름 관리 경험
- ✓ AI API 연동과 사용자 정품질 개선 반복 경험
- ✓ Local Storage 설계로 데이터 지속성 유지 방법 학습

참고 링크

https://github.com/eeeunhey/balanceeat

Daily Emotion Log — 감정 기록

2025.12 (팀) · HTML · CSS · JavaScript · Flask (Python) · MariaDB · Google Gemini API · Cloudtype



개요

하루를 마무리하며 감정을 기록하고, AI 기반 피드백과 패턴 시각화로 자기 이해를 돕는 웹 서비스입니다.

개발 기간/인원

2025.12 (1개월) · 5인 팀 / 담당: PM / Full-stack

목적 및 역할

- PM으로서 요구사항 정리 및 일정 관리
- Flask 백엔드 API 설계 및 구현
- Gemini API 연동으로 감정 분석 기능 구현
- Cloudtype 배포 설정

핵심기능

- 감정 선택 + 자유 메모 입력 UI
- AI(Gemini API)를 통한 감정 분석 및 피드백
- 날짜별 감정 시각화 (차트 & 타임라인)
- 세션 리포트 및 감정 트렌드 확인

기술스택

HTML/CSS/JS · Python · Flask · MariaDB · Gemini API · Cloudtype

성과·배운 점

- ✓ **PM 공식 역할**으로 팀원 일정 및 이슈 비의선 조율 경험
- ✓ Flask + MariaDB 연동으로 Python 백엔드 Full-stack 구현 경험
- ✓ Cloudtype 클라우드 배포로 실제 운영 환경 구성 학습

참고 링크

https://github.com/eeeunhey/daily-emotion-log

자기소개서

개발을 선택한 이유

대학에서 식품영양학을 전공하고 졸업한 뒤, 쿠팡 물류센터에서 전산·관리 업무를 약 3년간 담당했습니다. 재고 데이터를 정리하고, 출고 흐름을 점검하고, 단기 인력의 배치와 스케줄을 조율하는 일이 주된 업무였습니다. 매일 반복되는 수작업 속에서 **"이 작업을 자동화할 수 있다면 훨씬 정확하고 빠를 텐데"**라는 생각이 자주 스쳤고, 그것이 개발에 관심을 갖게 된 첫 계기였습니다.

막연한 관심에서 실제 전환을 결심하는 데는 용기가 필요했습니다. 비전공자라는 불안감이 있었지만, KDT 국비 과정에서 Java·Python 기초를 다지고 성남 폴리텍 하이테크 과정에서 Spring Boot와 React 기반의 실습 프로젝트를 진행하면서 **"모르는 것을 배워서 직접 만들어내는 과정"** 자체가 저와 잘 맞는다는 확신이 생겼습니다. 처음 내 손으로 만든 웹 페이지가 브라우저에서 동작하는 순간, 이 길이 맞다고 느꼈습니다. 현재는 고려사이버대학교 AI 데이터과학부에 편입해 이론적 기반도 병행하며 준비하고 있습니다.

나만의 학습 방식

새로운 기술을 접할 때 저는 튜토리얼을 따라 치는 것보다 **"왜 이 기술이 필요한지"**부터 이해하려 합니다. 목적을 먼저 파악하면 구조가 보이고, 응용할 때도 방향을 잡기가 훨씬 수월합니다. 모르는 부분이 생기면 공식 문서와 에러 메시지를 먼저 읽고, 그래도 해결이 안 되면 같은 상황을 겪은 사람들의 기록을 찾아가며 직접 원인을 파악하려 합니다. 이 습관이 쌓이면서 새로운 기술을 만나도 **겁부터 나기보다 어떻게 파고들지를 먼저 생각**하게 되었습니다.

Ascend(이력서 코칭 웹) 프로젝트를 예로 들면, 처음엔 props drilling으로 데이터를 전달했지만 화면이 복잡해질수록 어디서 상태가 바뀌는지 추적이 어려워졌습니다. 그래서 **Zustand로 전역 상태를 분리**하고, 서버 데이터는 React Query로 관리하는 구조로 리팩토링했습니다. "왜 나눠야 하는지"를 직접 겪고 나니, 라이브러리 선택 기준이 단순 유행에서 **프로젝트 맥락에 따른 판단**으로 바뀌었습니다.

ALBAIT(콘솔 구인·구직) 프로젝트에서는 SQL을 로직 안에 직접 작성하다 보니 기능이 늘수록 수정 범위가 예측 불가능해졌습니다. 이를 계기로 **DAO/DTO 패턴을 도입**해 데이터 접근 계층을 분리했고, "데이터가 어디서 만들어져서 어디로 전달되는지"가 코드만 봐도 보이는 구조를 만들 수 있었습니다. 이런 경험들이 쌓이면서 저는 **좋은 코드란 "동작하는 코드"가 아니라 "이해할 수 있는 코드"**라는 생각을 갖게 되었습니다.

협업과 책임감

7개의 프로젝트 중 4개를 팀으로 진행했습니다. Interview Simulator에서는 프론트엔드를 맡으면서 백엔드 팀원과 함께 API 명세를 먼저 정리했고, Daily Emotion Log에서는 PM으로 5인 팀의 일정과 역할 분배를 조율했습니다. 팀으로 일하면서 가장 크게 배운 건, **소통이 기술만큼 중요하다**는 점이었습니다. 서로 다른 방식으로 일하는 팀원들과 방향을 맞추고, 막히는 부분을 함께 풀어나가는 경험이 코드를 짜는 것 못지않게 값진 시간이었습니다.

팀으로 일하면서 알게 된 것이 있습니다. 같은 문제도 사람마다 전혀 다른 방식으로 풀어간다는 것입니다. 그 과정을 곁에서 보다 보면 자연스럽게 배우게 되고, 막힌 부분을 함께 살펴보다가 방향이 잡히는 순간에는 같이 기뻤습니다. 저는 **팀 전체가 같은 방향을 보며 막히지 않고 나아가는 것**이 결국 더 완성도 높은 결과물을 만든다고 생각합니다.

BalanceEat 프로젝트에서 PO를, Daily Emotion Log에서 PM을 맡았을 때는 솔직히 많이 어려웠습니다. 개발 업무와 달리 **"보이지 않는 것을 그려내는 일"**이었기 때문입니다. 요구사항이 모호한 상태에서 기획 방향을 정해야 했고, 팀원들이 각자 다르게 이해하고 있다는 걸 한참 지나서야 알아차린 적도 있었습니다. 소통을 더 자주, 더 명확히 해야 한다는 걸 경험으로 몸소 깨달은 시간이었습니다.

이 한계를 극복하기 위해 AI Studio와 프롬프트 기반 프로토타이핑 툴로 먼저 화면의 틀을 시각화한 뒤 팀원들과 공유하는 방식을 시도했습니다. 완벽하지는 않았지만, **"말로 설명하는 것"보다 "보여주는 것"이 훨씬 빠르게 맞춰진다**는 것을 확인했습니다.

돌아보면 가장 어려웠던 건 기획 도구가 아니라 **소통 그 자체**였습니다. 상대방의 말을 듣고 있었지만, 내 방식대로 해석하고 있었다는 걸 뒤늦게 알아차렸습니다. 그 결과로 방향이 어긋났을 때, 저는 처음으로 이런 질문을 하게 되었습니다. **"나는 최선을 다했는가, 아니면 나를 위한 최선을 다했는가?"** 그 차이가 팀 안에서는 생각보다 훨씬 크다는 걸, 그때 처음 실감했습니다.

조급하거나 불안한 상황일수록 시야가 좁아진다는 것도 알게 되었습니다. 그럴 때일수록 오히려 한 발 물러서서 "팀이 지금 무엇을 필요로 하는가"를 먼저 파악해야 한다는 걸 그 경험이 가르쳐 주었습니다. 현재는 《비폭력 대화》(마샬 B. 로젠버그)를 읽으며 **"상대방이 한 말이 아니라, 하고 싶었던 말을 듣는 것"**에 대해 공부하고 있습니다. 이 경험이 없었다면 내가 무엇을 모르는지조차 알지 못했을 것입니다. **좌절했기 때문에, 오히려 더 단단한 방향으로 성장할 수 있었습니다.**

자기소개서 (계속)

기술 적용 경험

프로젝트를 거듭하면서 가장 크게 배운 건, 기술 자체보다 **어떤 문제를 풀기 위해 그 기술을 썼는지**가 더 중요하다는 점이었습니다.

TypeScript 도입으로 유지보수 속도 개선

Ascend와 ResumeBuilder를 처음 만들 때는 JavaScript만으로 충분하다고 생각했습니다. 그런데 컴포넌트가 20개를 넘기면서 문제가 생기기 시작했습니다. 어떤 컴포넌트에 어떤 props를 넘겨야 하는지, API에서 받아온 데이터의 구조가 어떤 형태인지 매번 콘솔에 찍어보거나 다른 파일을 열어서 확인해야 했습니다. 간단한 수정 하나에도 관련 파일을 3~4개씩 뒤져야 하는 상황이 반복되었습니다.

TypeScript를 도입한 뒤 가장 먼저 달라진 건 **IDE가 코드의 맥락을 알려준다**는 점이었습니다. API 응답 타입을 interface로 한 번 정의해두니, 그 데이터를 사용하는 모든 곳에서 자동완성이 되었고 구조가 바뀌면 빌드 전에 에러가 띄워졌습니다. 특히 여러 화면에서 같은 데이터를 다르게 해석하던 문제가 사라지면서, **"이 값이 어디서 왔고, 어떤 형태인지"를 코드만 보고 바로 파악**할 수 있게 되었습니다. 돌이켜보면 TypeScript는 단순한 타입 검사 도구가 아니라, 팀원이 없는 개인 프로젝트에서 **과거의 내가 미래의 나에게 남기는 문서** 같은 역할을 해주었습니다.

Zustand 전역 스토어로 상태 불일치 개선

Ascend 프로젝트에서 이력서 업로드 → AI 분석 → 결과 표시 → 코칭 피드백까지의 흐름을 만들다 보니, 분석 결과를 보여주는 화면, 수정 입력을 받는 화면, 최종 요약 화면이 각각 따로 상태를 들고 있었습니다. 한 화면에서 데이터를 수정해도 다른 화면에는 반영이 안 되거나, 뒤로 갔다 돌아오면 입력값이 초기화되는 문제가 계속 생겼습니다. 처음에는 props로 넘기면 해결되겠지 싶었지만, 중간 컴포넌트가 많아지면서 drilling이 5단계까지 내려가는 상황이 되었습니다.

Zustand를 도입해 분석 결과, 사용자 입력, UI 상태를 **각각의 store로 나눠서 한 곳에서 관리**하도록 바꿨습니다. 각 컴포넌트는 자기가 필요한 값만 selector로 구독하게 만들었더니, 불필요하게 전체가 리렌더링되던 현상이 눈에 띄게 줄었고 버그가 생겼을 때도 "어느 store의 어떤 액션에서 값이 바뀌었는지"를 바로 추적할 수 있었습니다. 이 경험을 통해 **상태 관리는 도구 선택보다 "무엇을 전역으로 올릴 것인가"에 대한 판단**이 더 중요하다는 것을 체감했습니다.

DAO/DTO 패턴으로 DB 연동 구조 개선

ALBAIT은 Java 콘솔 기반으로 만든 구인·구직 매칭 프로젝트였습니다. 처음에는 서비스 로직 안에 SQL 쿼리를 직접 작성했는데, 회원 관리, 채용 공고, 지원 내역 세 가지 도메인이 하나의 클래스에 뒤섞이다 보니 한 기능을 수정하면 예상치 못한 곳에서 오류가 발생하는 일이 잦아졌습니다. 예를 들어, 공고 목록 조회를 고쳤더니 지원 내역 화면에서 NullPointerException이 터지는 식이었습니다.

이를 해결하기 위해 **DAO로 데이터베이스 접근을, DTO로 계층 간 데이터 전달을 분리**했습니다. 회원DAO, 공고DAO, 지원DAO로 나누고, 각 DAO는 자기 테이블만 책임지도록 했습니다. 서비스 로직에서는 DTO 객체를 통해 필요한 데이터만 주고받게 되면서, **"어떤 데이터가 어디서 만들어져서 어디로 가는지"**가 코드 구조만 보고도 파악할 수 있게 되었습니다. 이후 "인기 공고 추천" 기능을 추가할 때도 기존 코드 수정 없이 새 DAO 메서드만 추가해서 해결할 수 있었고, **구조를 나누는 것이 기능을 빼는 게 아니라 오히려 확장을 쉽게 만든다**는 걸 직접 경험했습니다.

앞으로의 다짐

지난 2년간 7개의 프로젝트를 통해 프론트엔드(React, TypeScript)부터 백엔드(Spring Boot, JPA), AI 연동(LLM, Object Detection)까지 직접 구현해보았습니다. 물론 아직 실무 경험이 부족한 주니어입니다. 하지만 **모르는 것을 인정하고 빠르게 배우는 것**, 그리고 **막히는 상황에서 포기 대신 우회 경로를 찾는 것**은 자신 있습니다.

쿠팡에서 3년간 물류 현장을 경험하며 익힌 업무 이해력과 커뮤니케이션 역량, 그리고 비전공에서 출발해 스스로 기술 스택을 쌓아온 자기 주도적 학습 능력을 바탕으로, **팀에 빠르게 적응하고, 맡은 일을 끝까지 완성하는 개발자**가 되겠습니다.

부족한 부분은 빠르게 채워가며, **함께 일하고 싶은 동료**로 기억되는 개발자가 되겠습니다.